Open in app ↗

Medium

Member-only story

Claude Code Permission Hook: Uninterrupted Coding Without the Risk

4 min read · Feb 17, 2026

Code Coup Follow

Listen Share ••• More

There's a specific kind of frustration only developers understand. You're deep in flow. Claude is writing solid code. The architecture is clicking into place.

Then, *click approve.*

Again.

And again.

Twenty approvals later, your brain has context-switched so many times that the elegant solution you were building now feels distant.

This is the hidden productivity tax of AI-assisted development.

But what if Claude could operate independently **without putting your system at risk?**

That's exactly what the **Permission Hook** solves.

The Problem With Claude Code

Every developer using Claude Code eventually runs into the same dilemma.

You get **safety**, or you get **speed** — but rarely both.

Option 1: Approve Everything Manually

Safe? Yes. Efficient? Not even close. Complex features can trigger **50+ permission prompts**. Each one breaks your concentration, and flow state is notoriously hard to regain. Momentum disappears.

Option 2: Skip Permissions Entirely

Fast? Extremely. Safe? Absolutely not.

Using `--dangerously-skip-permissions` is like removing the brakes from a sports car because you want to drive faster.

One hallucinated command like:

```
rm -rf /
```

…and your machine is effectively gone. Fine for disposable experiments. Unacceptable for production work.

The Win

Install the Permission Hook and let Claude operate without constant interruptions:

```
npm install -g @abdo-el-mobayad/claude-code-fast-permission-hook
cf-approve install
cf-approve config
```

Three commands. That's it. Now Claude keeps working while dangerous operations are automatically blocked — no risky flags required.

The Smarter Approach: Intelligent Delegation

Instead of choosing between paranoia and recklessness, the Permission Hook introduces a third model:

Delegated intelligence.

Claude runs freely. Threatening commands get blocked instantly Ambiguous cases are reviewed by a lightweight AI.

Think of it as installing a **real-time security reviewer** between Claude and your machine. No micromanagement required.

How It Works: The Three-Tier Decision System

When Claude requests permission, the hook evaluates it immediately using a layered strategy.

Tier 1 — Fast Approve (No AI Needed)

Safe operations pass through without delay:

```
Read, Glob, Grep, WebFetch, WebSearch
Write, Edit, MultiEdit, NotebookEdit
TodoWrite, Task, all MCP tools
```

No latency. No API cost. No interruptions. Claude simply continues working. This is where **most operations land**, which is why the hook feels invisible during daily use.

Tier 2 — Fast Deny

Some commands are too dangerous to evaluate dynamically.

They never execute.

```
# These never execute, period
rm -rf /                        # System destruction
git push --force origin main  # Protected branch overwrite
mkfs /dev/sda                   # Disk formatting
:(){ :|:& };:                    # Fork bomb
```

No AI involved. No debate. Just hard protection against catastrophic mistakes. It's the equivalent of having guardrails on a mountain road.

Tier 3 — LLM Analysis (Cached Decisions)

Not every command is obviously safe or dangerous.

Consider something like cleaning Docker resources.

Is it maintenance — or accidental data loss?

The hook sends context to a fast, low-cost LLM for evaluation:

```
{
  "tool": "Bash",
  "command": "docker system prune -af",
  "working_directory": "/home/user/project",
  "recent_context": "User asked to clean up Docker resources"
}
```

The model understands **intent**, not just syntax. Even better — decisions are cached. Run the same command again? Instant approval. No extra latency. No extra cost.

Configuration

The hook stores its settings here:

```
~/.claude-code-fast-permission-hook/config.json
```

Example configuration:

```
{
  "llm": {
    "provider": "openai",
    "model": "openai/gpt-4o-mini",
    "apiKey": "sk-or-v1-your-key",
    "baseUrl": "https://openrouter.ai/api/v1"
  },
  "cache": {
    "enabled": true,
    "ttlHours": 168
  }
}
```

Why OpenRouter? Latency. Lower cost. Broad model compatibility.

The estimated cost is roughly **$1 for 5,000+ decisions**, but in real-world usage, Tier 1 and Tier 2 handle most operations.

Translation: You might spend a dollar every few months.

Installation Levels: Choose Once or Customize

The hook supports two installation scopes.

Device Level (Recommended)

Configure it once:

```
~/.claude/settings.json
```

It applies everywhere. Perfect for developers who want a “set it and forget it” workflow.

Project Level

Need stricter rules for production infrastructure?

Use:

```
.claude/settings.local.json
```

Great for teams, shared repos, or sensitive environments.

What the Installer Adds Behind the Scenes

The installer wires the permission system directly into Claude:

```
{
  "hooks": {
    "PermissionRequest": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "cf-approve permission"
          }
        ]
      }
    ]
  }
}
```

Every permission request now routes through the hook automatically. No extra steps required from you.

When Things Go Wrong

Even great tools occasionally need troubleshooting.

Error: “Permission denied” on all operation

Your API key is likely missing or invalid.

Fix it quickly:

```
cf-approve config
```

Re-enter your OpenRouter key.

Error: “Hook not triggering”

Verify installation:

```
cf-approve doctor
cf-approve status
```

Behavior Seems Inconsistent

Clear the cache:

```
cf-approve clear-cache
```

Simple reset. Problem solved.

Why This Matters More Than You Think

AI coding tools are evolving rapidly.

But developer experience is still fragile.

Small interruptions compound into massive productivity drains.

The Permission Hook removes one of the biggest hidden frictions in AI-assisted development:

Approval fatigue.

Instead of babysitting Claude, you return to what actually matters: Designing systems. Solving problems. Shipping faster.

The Bigger Picture: Foundational Hooks

The Permission Hook is only half of a larger philosophy inside **ClaudeFast**.

The other half?

Skill Activation Hook — ensuring Claude loads the right capabilities exactly when needed.

Together they create something powerful:

- You type naturally
- Claude executes efficiently
- The framework orchestrates everything silently

No friction. No constant supervision. Just flow.

Final Thoughts

Great tooling should disappear into the background.

You shouldn't have to think about permissions any more than you think about your keyboard while typing.

The Permission Hook moves AI-assisted coding closer to that ideal — where safety is automatic, speed is preserved, and your focus stays exactly where it belongs. On building. Because the future of development isn't about controlling AI every second. It's about trusting the right guardrails to move faster than ever.

Claude Code | Claude Code Hooks | AI | Ai Coding | Coding

Code Coup

Follow

Written by Code Coup

3.8K followers · 1 following

Code Coup: Seize the Code, Stage a Coup!

Responses (1)

Holger Kreissl

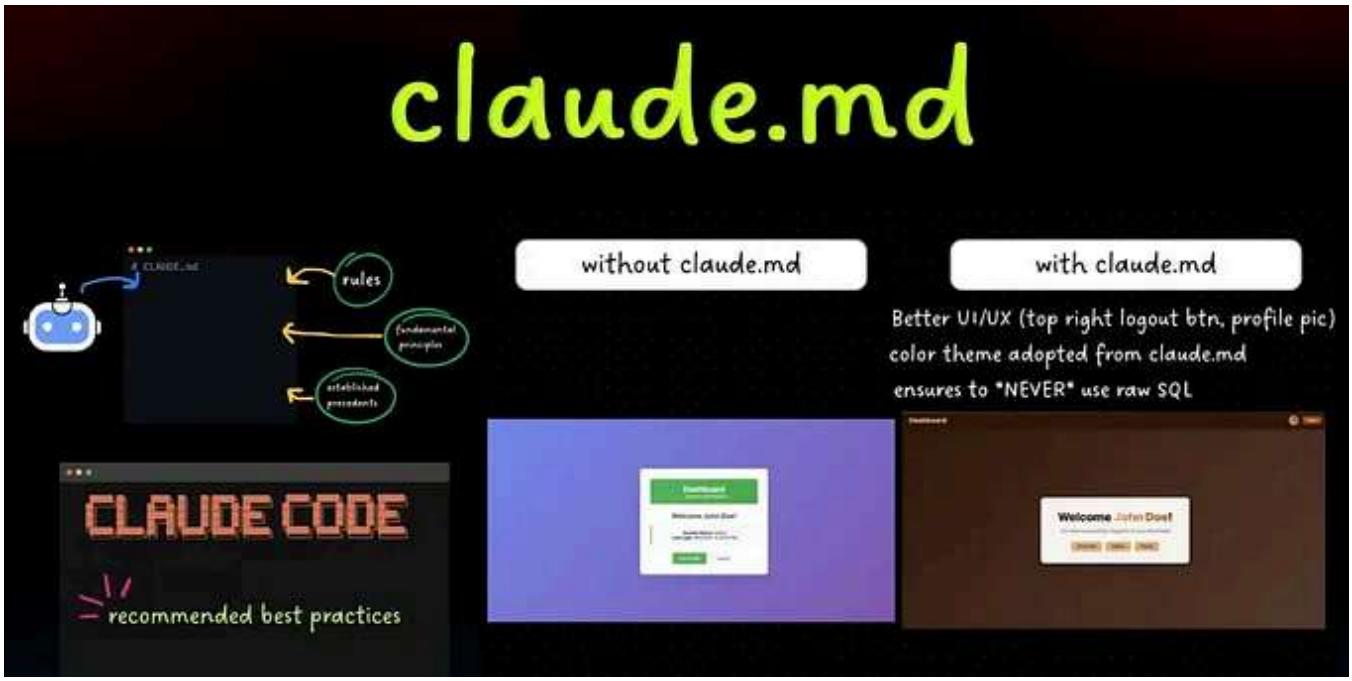
What are your thoughts?

Sam Miller
Feb 18

Link to permission hook?

Reply

More from Code Coup



In Coding Nexus by Code Coup

I Fixed AI Coding With One File — Karpathy-inspired CLAUDE.md

Karpathy-inspired Claude Code rules that stop overengineering, reduce mistakes, and make AI write clean, minimal code.

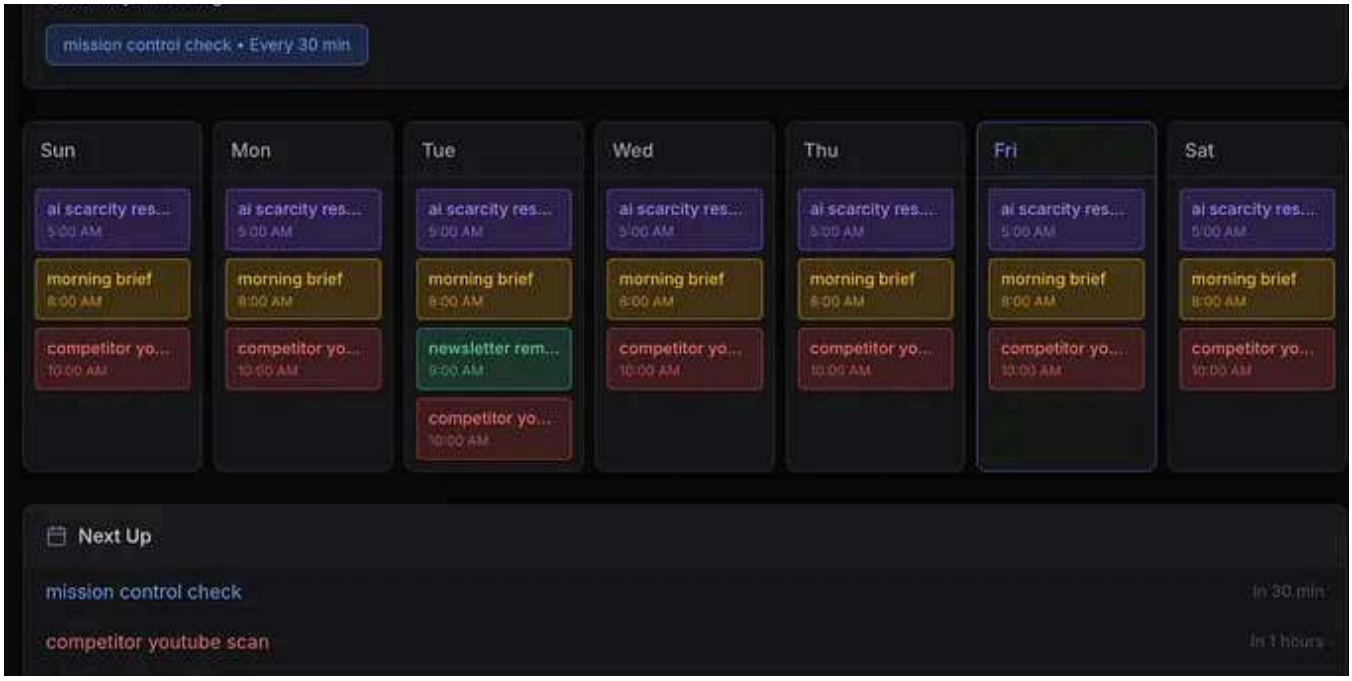
Jan 31 155 2

In Coding Nexus by Civil Learning

I Built an AI Company with OpenClaw + Vercel + Supabase — Two Weeks Later, They Run It Themselves

6 AI agents, 1 VPS, 1 Supabase database — the journey from “agents can talk” to “agents run the website autonomously”.

Feb 11 519 18

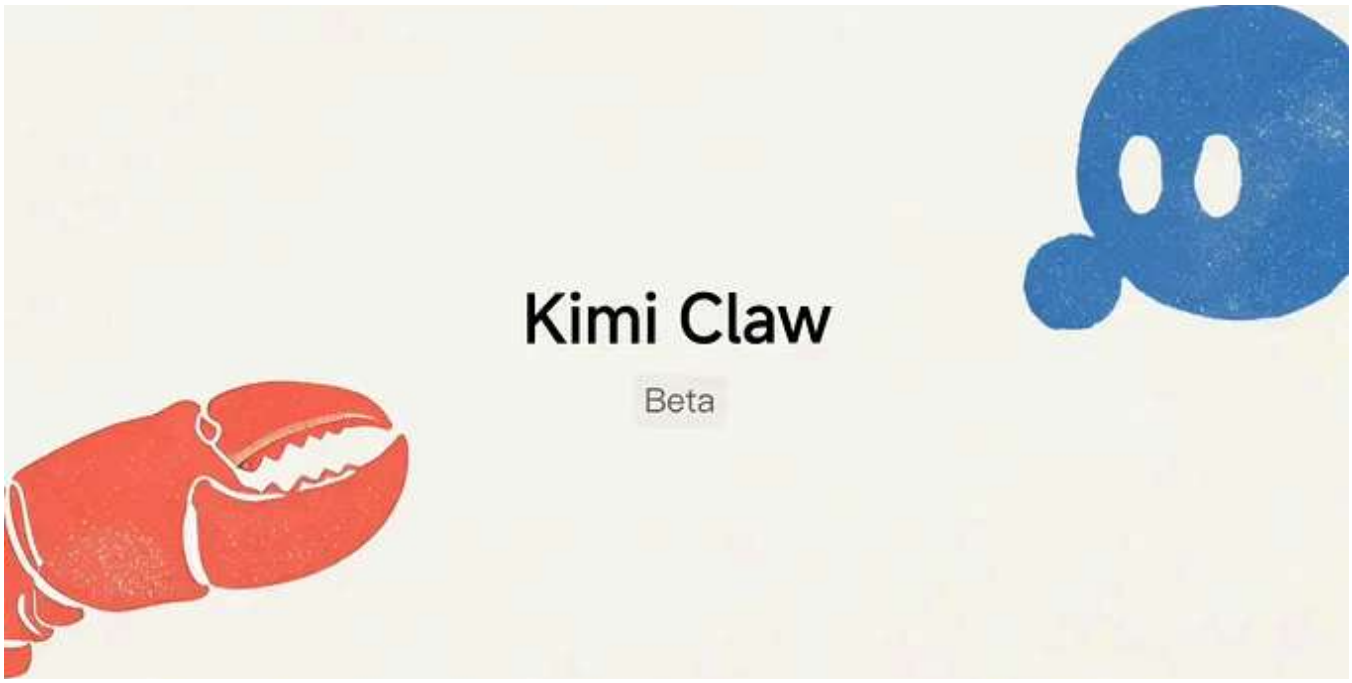


In Coding Nexus by Civil Learning

3 Things You Must Build Immediately With OpenClaw (If You Want Real Autonomy)

If you’re running OpenClaw seriously, there’s a moment you eventually hit.

Feb 10 335 1



Code Coup

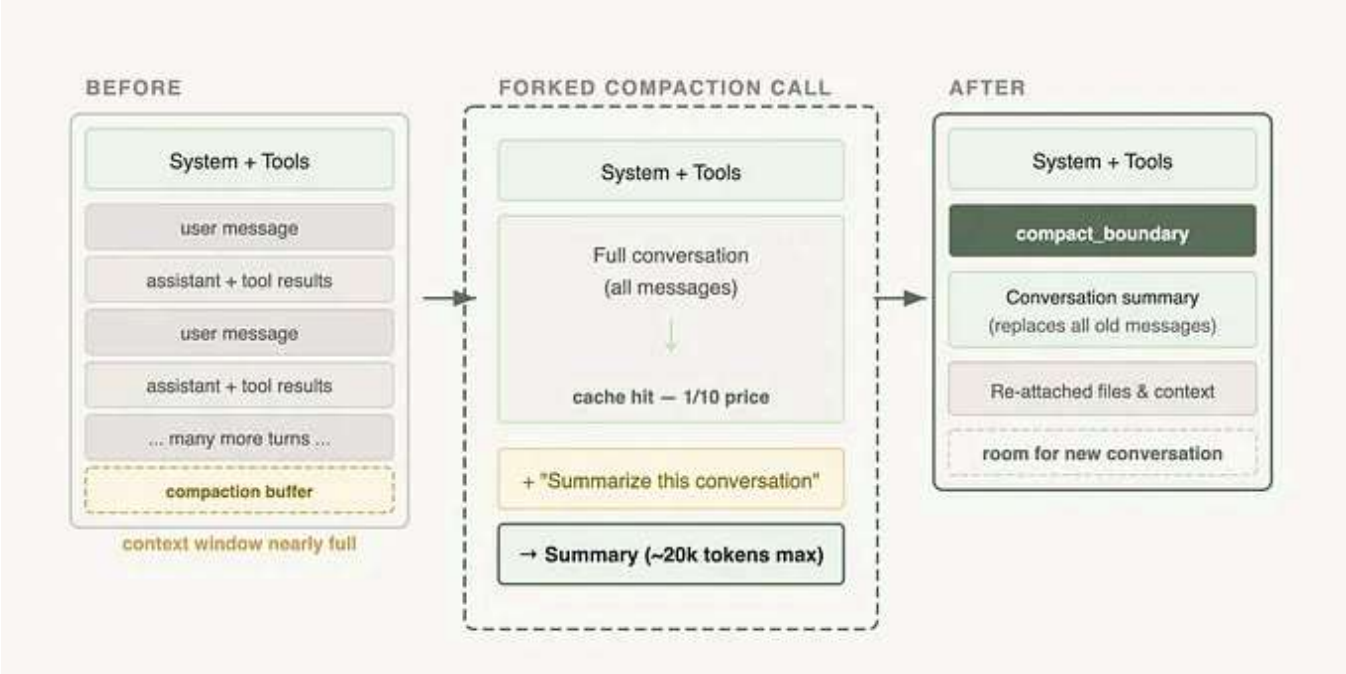
Kimi Claw: Kimi Just Put OpenClaw in Your Browser

OpenClaw is now native to kimi.com. Living right in your browser tab. Online 24/7. No server setup. No command-line nonsense. Just... there.

Feb 15 73 2

See all from Code Coup

Recommended from Medium

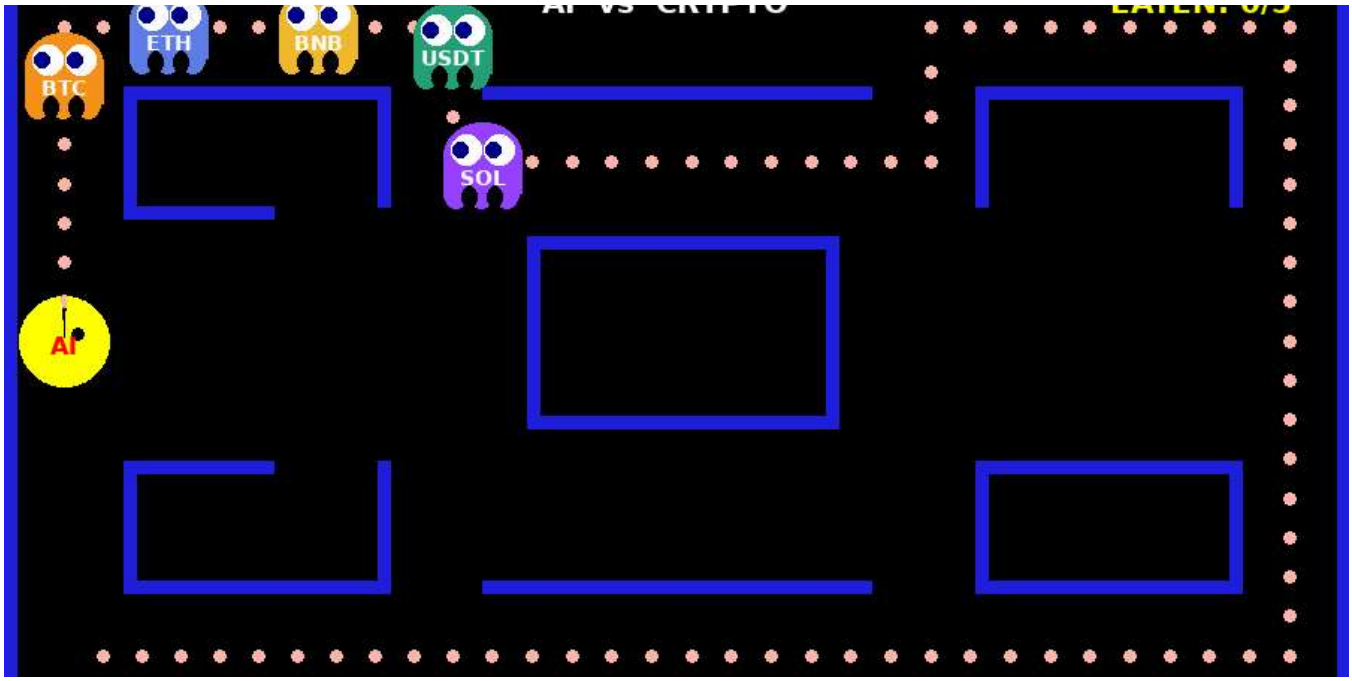


In AI Software Engineer by Joe Njenga

Anthropic Just Fixed the Biggest Hidden Cost in AI Agents (Automatic Prompt Caching)

With just one change, you can cut Claude API costs to 10 cents on the dollar and stop bleeding cash on every single API call.

Feb 20 741 16

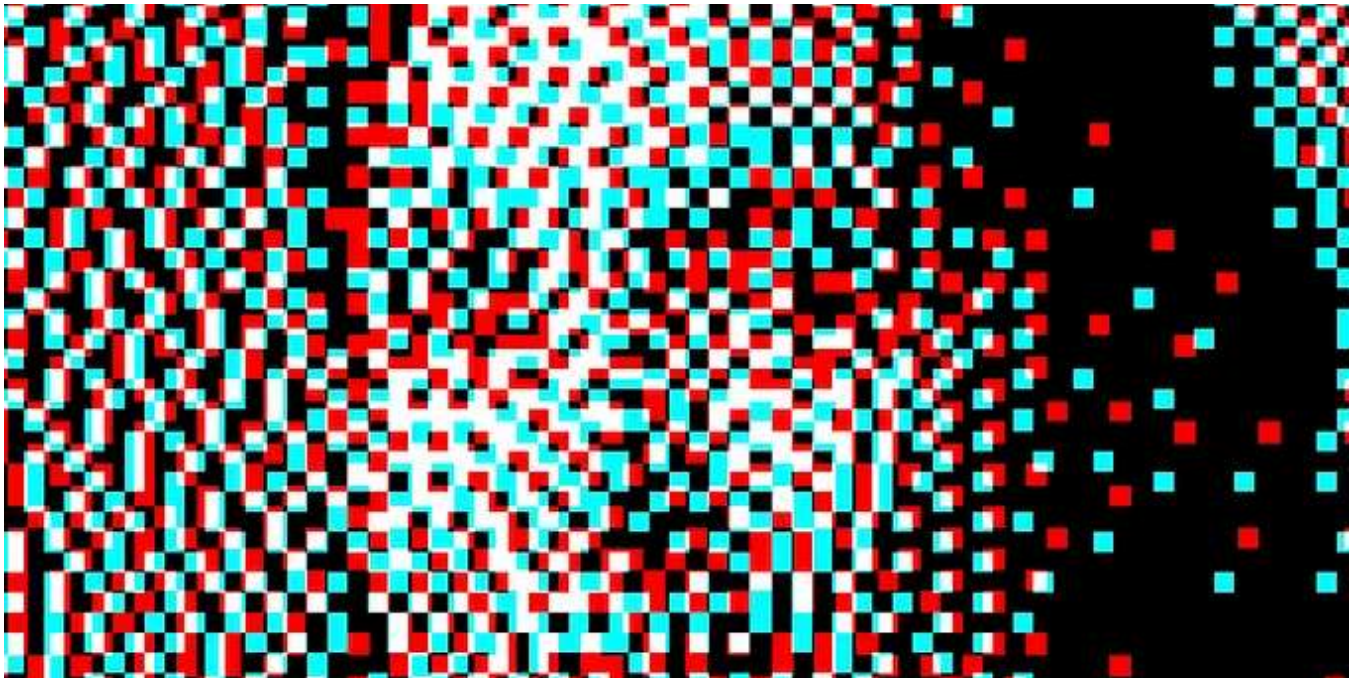


In AI Advances by Jose Crespo, PhD

Anthropic is Killing Bitcoin

The AI-native currency already exists — hiding in plain sight, outperforming crypto by six orders of magnitude.

Feb 17 2.7K 123

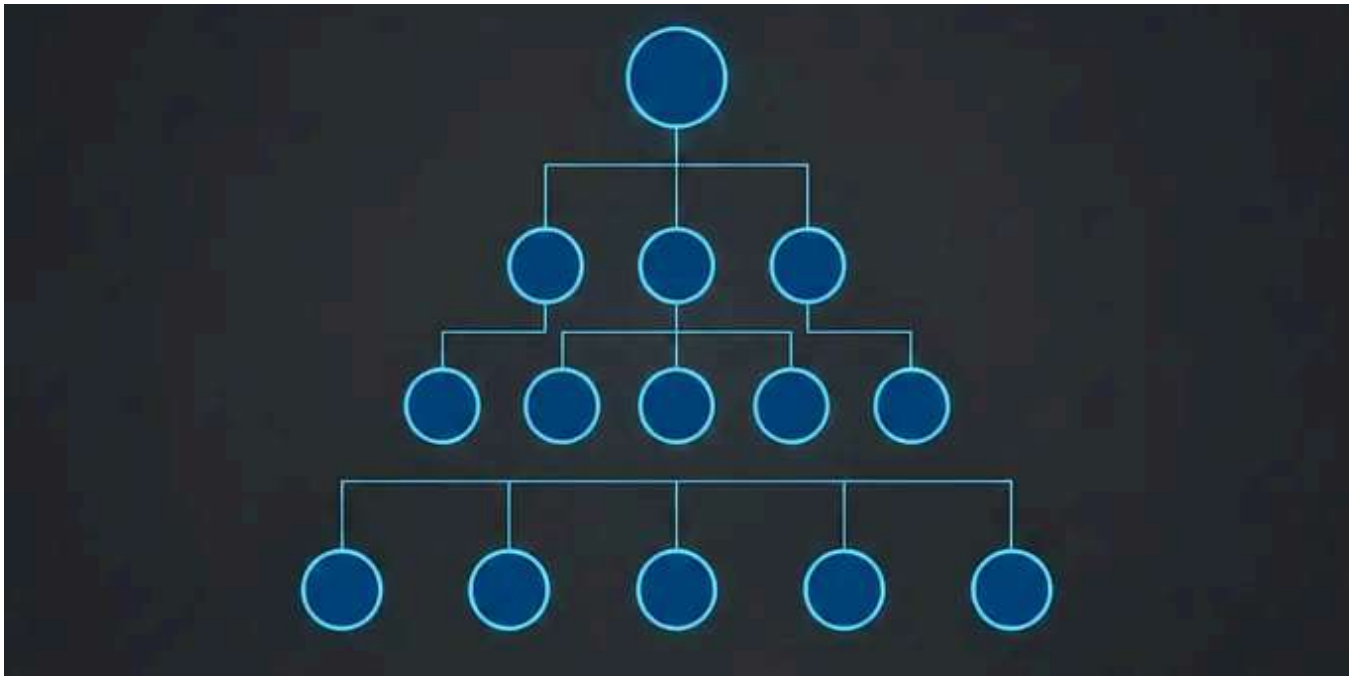


Marco Kotrotsos

The Agentic Engineering Playbook.

According to OpenClaw's Creator

Feb 17 212 8

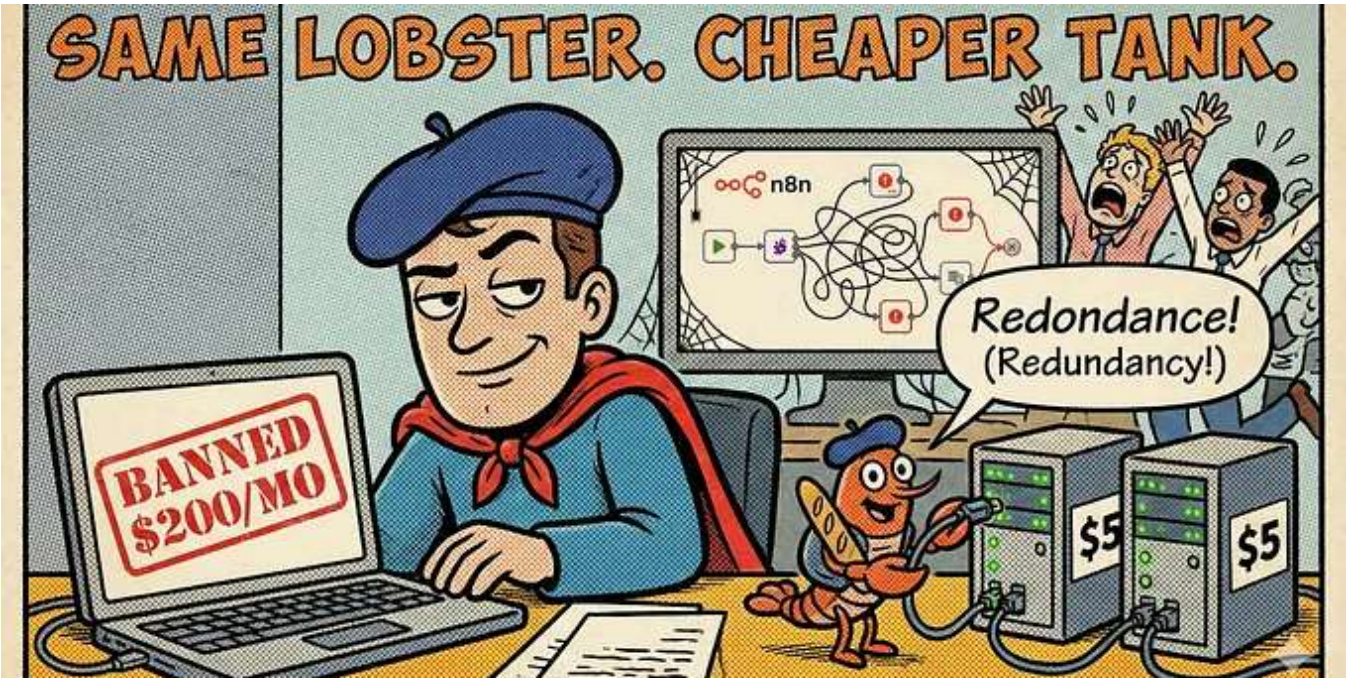


Reza Rezvani

OpenClaw Multi-Agent System: The Blueprint I Built in 12 Hours

From one AI Assistant built with OpenClaw (Moltbot) 13 specialized agents with cron jobs, shared memory, and self-updating kanban — the...

Feb 17 437 9



Phil | Rentier Digital Automation

Anthropic Just Killed My $200/Month OpenClaw Setup. So I Rebuilt It for $15.

My OpenClaw instance ran on Claude Max for six weeks. Emails, calendar, Telegram, the whole personal Jarvis fantasy. Then one morning...

Feb 19 765 28

In Write A Catalyst by NAJEEB

I Burned Millions of Tokens on Claude Code. Here Is Why “Vibe Coding” Is a Trap.

I have spent the last couple of months living inside Claude Code. I have used it to build complex graphics, internal tooling, and several...

Feb 11 466 22

See more recommendations